Cryptography

University of Insubria

Cryptography

Cryptography is the practice and study of techniques for secure (secret) communication in the presence of third parties called opponents

It is possible to obtain

- Confidentiality
- Integrity
- Authentication
- Non-repudiation

Cryptography

- Confidentiality:

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message

- **Integrity**:

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message

- **Integrity**: if the message has been modified during transmission, it must be possible to detect this

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message
- **Integrity**: if the message has been modified during transmission, it must be possible to detect this
- **Authentication**:

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message
- **Integrity**: if the message has been modified during transmission, it must be possible to detect this
- **Authentication**: the identity of the parties in a communication must be verifiable

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message
- **Integrity**: if the message has been modified during transmission, it must be possible to detect this
- **Authentication**: the identity of the parties in a communication must be verifiable
- **Non-repudiation**:

Cryptography

- **Confidentiality**: no one outside the communication can read the contents of the message
- **Integrity**: if the message has been modified during transmission, it must be possible to detect this
- **Authentication**: the identity of the parties in a communication must be verifiable
- **Non-repudiation**: if a person sends a specific message, it must not be possible for him to deny having done so

Definitions

- **Cipher**: algorithm for performing operations to obscure (encrypt) a plaintext (encryption), or to restore a previously encrypted message (decryption)
    - **symmetric**: cipher that uses the same key to encrypt and decrypt
    - **asymmetric**: cipher that uses two keys

- **Key**: secret used in the cipher to encrypt or decrypt messages

Kerckhoffs/Shannon Principle

The security of a cryptographic system should not depend on keeping the cryptographic algorithm hidden but only on keeping the key hidden

Challenge

Crypto 01 - Encoding 1

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 01 - Encoding 1

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	*[NULL]*
1	1	1	1	*[START OF HEADING]*
2	2	10	2	*[START OF TEXT]*
3	3	11	3	*[END OF TEXT]*
4	4	100	4	*[END OF TRANSMISSION]*
5	5	101	5	*[ENQUIRY]*
6	6	110	6	*[ACKNOWLEDGE]*
7	7	111	7	*[BELL]*
8	8	1000	10	*[BACKSPACE]*
9	9	1001	11	*[HORIZONTAL TAB]*
10	A	1010	12	*[LINE FEED]*
11	B	1011	13	*[VERTICAL TAB]*
12	C	1100	14	*[FORM FEED]*
13	D	1101	15	*[CARRIAGE RETURN]*
14	E	1110	16	*[SHIFT OUT]*
15	F	1111	17	*[SHIFT IN]*
16	10	10000	20	*[DATA LINK ESCAPE]*
17	11	10001	21	*[DEVICE CONTROL 1]*
18	12	10010	22	*[DEVICE CONTROL 2]*
19	13	10011	23	*[DEVICE CONTROL 3]*
20	14	10100	24	*[DEVICE CONTROL 4]*
21	15	10101	25	*[NEGATIVE ACKNOWLEDGE]*
22	16	10110	26	*[SYNCHRONOUS IDLE]*
23	17	10111	27	*[END OF TRANS. BLOCK]*
24	18	11000	30	*[CANCEL]*
25	19	11001	31	*[END OF MEDIUM]*
26	1A	11010	32	*[SUBSTITUTE]*
27	1B	11011	33	*[ESCAPE]*
28	1C	11100	34	*[FILE SEPARATOR]*
29	1D	11101	35	*[GROUP SEPARATOR]*
30	1E	11110	36	*[RECORD SEPARATOR]*
31	1F	11111	37	*[UNIT SEPARATOR]*
32	20	100000	40	*[SPACE]*
33	21	100001	41	!
34	22	100010	42	"
35	23	100011	43	#
36	24	100100	44	$
37	25	100101	45	%
38	26	100110	46	&
39	27	100111	47	'
40	28	101000	50	(
41	29	101001	51	)
42	2A	101010	52	*
43	2B	101011	53	+
44	2C	101100	54	,
45	2D	101101	55	-
46	2E	101110	56	.
47	2F	101111	57	/
48	30	110000	60	**0**
49	31	110001	61	**1**
50	32	110010	62	**2**
51	33	110011	63	**3**
52	34	110100	64	**4**
53	35	110101	65	**5**
54	36	110110	66	**6**
55	37	110111	67	**7**
56	38	111000	70	**8**
57	39	111001	71	**9**
58	3A	111010	72	:
59	3B	111011	73	;
60	3C	111100	74	<
61	3D	111101	75	=
62	3E	111110	76	>
63	3F	111111	77	?
64	40	1000000	100	@
65	41	1000001	101	**A**
66	42	1000010	102	**B**
67	43	1000011	103	**C**
68	44	1000100	104	**D**
69	45	1000101	105	**E**
70	46	1000110	106	**F**
71	47	1000111	107	**G**
72	48	1001000	110	**H**
73	49	1001001	111	**I**
74	4A	1001010	112	**J**
75	4B	1001011	113	**K**
76	4C	1001100	114	**L**
77	4D	1001101	115	**M**
78	4E	1001110	116	**N**
79	4F	1001111	117	**O**
80	50	1010000	120	**P**
81	51	1010001	121	**Q**
82	52	1010010	122	**R**
83	53	1010011	123	**S**
84	54	1010100	124	**T**
85	55	1010101	125	**U**
86	56	1010110	126	**V**
87	57	1010111	127	**W**
88	58	1011000	130	**X**
89	59	1011001	131	**Y**
90	5A	1011010	132	**Z**
91	5B	1011011	133	[
92	5C	1011100	134	\
93	5D	1011101	135	]
94	5E	1011110	136	^
95	5F	1011111	137	_
96	60	1100000	140	`
97	61	1100001	141	**a**
98	62	1100010	142	**b**
99	63	1100011	143	**c**
100	64	1100100	144	**d**
101	65	1100101	145	**e**
102	66	1100110	146	**f**
103	67	1100111	147	**g**
104	68	1101000	150	**h**
105	69	1101001	151	**i**
106	6A	1101010	152	**j**
107	6B	1101011	153	**k**
108	6C	1101100	154	**l**
109	6D	1101101	155	**m**
110	6E	1101110	156	**n**
111	6F	1101111	157	**o**
112	70	1110000	160	**p**
113	71	1110001	161	**q**
114	72	1110010	162	**r**
115	73	1110011	163	**s**
116	74	1110100	164	**t**
117	75	1110101	165	**u**
118	76	1110110	166	**v**
119	77	1110111	167	**w**
120	78	1111000	170	**x**
121	79	1111001	171	**y**
122	7A	1111010	172	**z**
123	7B	1111011	173	{
124	7C	1111100	174	|
125	7D	1111101	175	}
126	7E	1111110	176	~
127	7F	1111111	177	*[DEL]*

Table Source: https://simple.m.wikipedia.org

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 01 - Encoding 1

2_crypto > crypto01.py > ...

```python
vec = [102, 108, 97, 103, 123, 117, 103, 104, 95, 78, 117, 109, 66, 51, 114, 53, 95, 52, 49, 114, 51, 52, 100, 121, 125]
for el in vec:
    value = chr(el)
    print(value, end="")

```

flag: flag{ugh_NumB3r5_41r34dy}

Challenge

Crypto 02 - Encoding 2

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 02 - Encoding 2

2_crypto > crypto02.py > ...

```python
string = "666c61677b68337834646563696d616c5f63346e5f62335f41424144424142457d"

b = bytes.fromhex(string)
print(b)

```

flag: flag{h3x4decimal_c4n_b3_ABADBABE}

Challenge

Crypto 03 - Encoding 3

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 03 - Encoding 3

2_crypto > crypto03.py > ...

```python
from base64 import b64decode

s1 = "ZmxhZ3t3NDF0XzF0c19hbGxfYjE="
s2 = 66481303558391800646274589843198128673763592972 5

s1 = b64decode(s1)
s2 = s2.to_bytes(20, 'big')

print(s1 + s2)

```

flag: flag{w41t_1ts_all_b1ts?_4lw4ys_H4s_b33n}

Symmetric encryption

$\mathcal{K}$: key space

$\mathcal{M}$: plaintext space

$\mathcal{C}$: ciphertext space

Encryption scheme

- $\text{Gen} \rightarrow \mathcal{K}$
- $\text{Enc} : \mathcal{M} \times \mathcal{K} \rightarrow \mathcal{C}$
- $\text{Dec} : \mathcal{C} \times \mathcal{K} \rightarrow \mathcal{M}$

Symmetric encryption

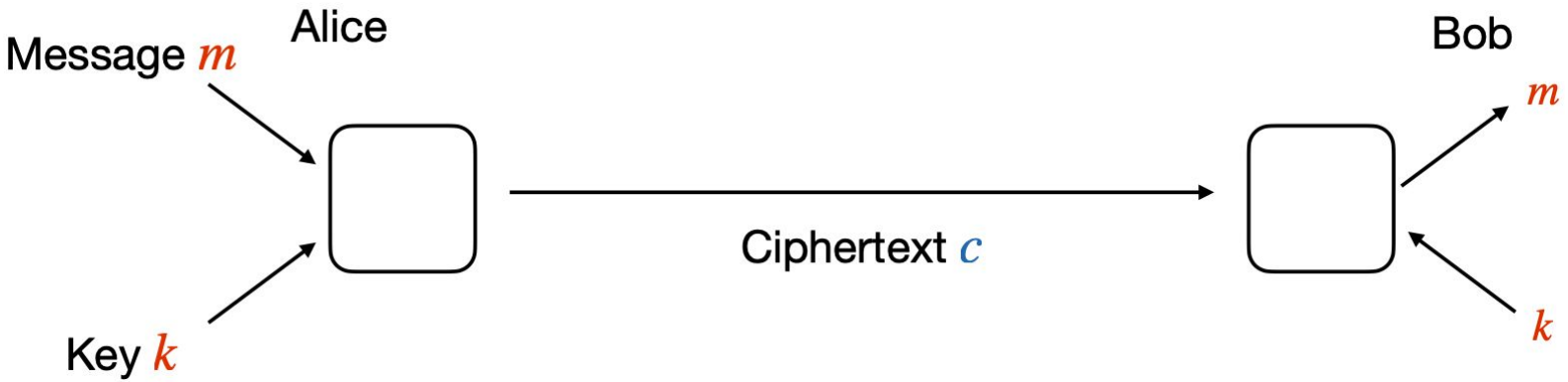
It is also known as private key or single key encryption. The secret key is known only to the sender and the recipient.

Correctness

- encrypted_message = Enc(message, key)
- message = Dec(encrypted_message, key)

For every $k \in \mathcal{K}$, for every $m \in \mathcal{M}$ $Dec_k(Enc_k(m)) = m$

Symmetric encryption



Sizes of key spaces

2^{48} (Mifare Crypto-1): your smartphone

2^{56} (DES): modern laptop

2^{90} (hashes/year of Bitcoin network): large data center

2^{128} (AES): reasonably secure

2^{256} (AES): quantum-secure

Sizes of key spaces

2^{48} (Mifare Crypto-1): your smartphone

2^{56} (DES): modern laptop

2^{90} (hashes/year of Bitcoin network): large data center

2^{128} (AES): reasonably secure <- most used

2^{256} (AES): quantum-secure

Sizes of key spaces

Why 128?

- Estimated Sun lifetime: 2^{60} secs = 10 billion years
- Suppose $2^{50} \approx 10^{15}$ keys are tested per sec
- To find the right 128-bit key requires 128,000 (= 217) sun lifetimes

Symmetric encryption

Cryptographic schemes differ in

- The type of operation in which a plaintext P is transformed into a ciphertext C
  - **Substitutions**: each element of P is replaced with another element
  - **Transpositions**: exchanges are made between elements of P

- Mode of "treatment" of P
  - **Block ciphers**: P is processed by the algorithm "one block at a time"
  - **Stream ciphers**: P is processed considering a single element at a time

Monoalphabetic Cipher

- An encryption technique based on substitution relies on replacing each element of the plaintext with another element from the same alphabet using a correspondence table. The correspondence table is the key.

Chiaro	Cifrato
a	f
b	q
c	t
d	r
...	...
z	g

Example

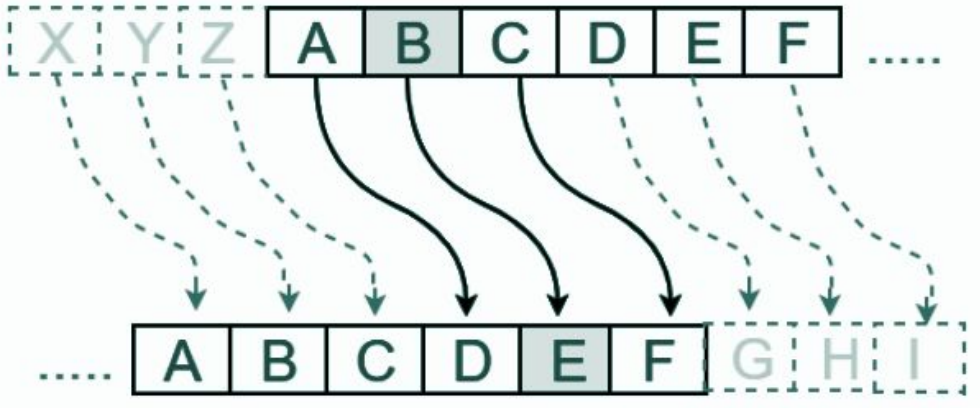
- Enc(abc, 5) = fqt
- Dec(fqt, 5) = abc

Monoalphabetic Cipher

- Julius Caesar: the algorithm requires replacing each letter of the alphabet with the one which is "three hops" away; when the alphabet ends it starts counting again.

Example

- Enc(HELLO) =

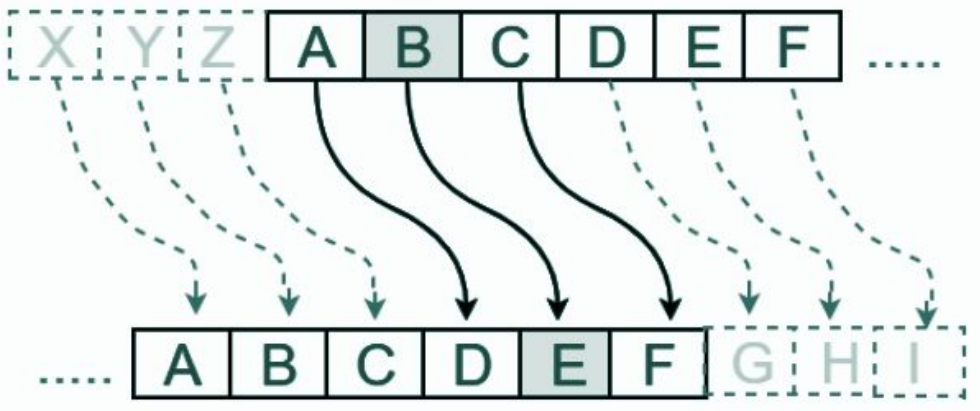


Monoalphabetic Cipher

- Julius Caesar: the algorithm requires replacing each letter of the alphabet with the one which is "three hops" away; when the alphabet ends it starts counting again.

Example

- Enc(HELLO) = KHOOR
- Dec(KHOOR) =

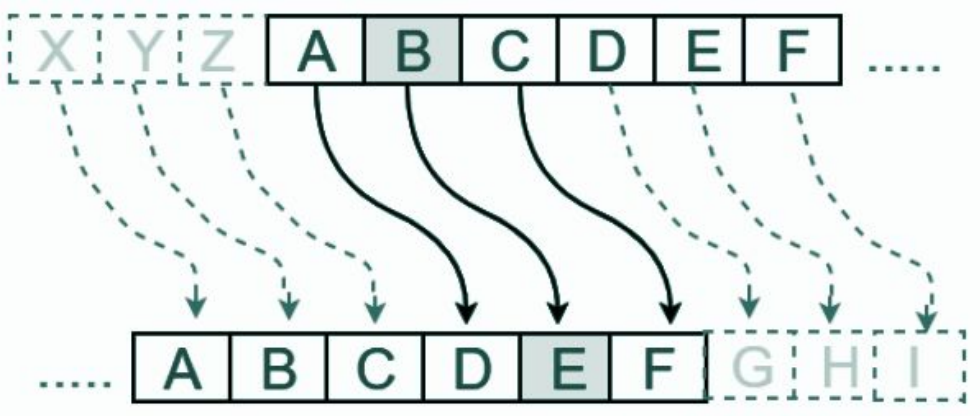


Monoalphabetic Cipher

- Julius Caesar: the algorithm requires replacing each letter of the alphabet with the one which is "three hops" away; when the alphabet ends it starts counting again.

Example

- Enc(HELLO) = KHOOR
- Dec(KHOOR) = HELLO



Cryptanalysis

Cryptanalysis is the study and practice of analyzing and breaking cryptographic security systems to gain access to encrypted information without authorization.

- finding weaknesses in encryption algorithms
- cracking codes and recovering plaintext

Attack types

-
-

Cryptanalysis

Cryptanalysis is the study and practice of analyzing and breaking cryptographic security systems to gain access to encrypted information without authorization.

- finding weaknesses in encryption algorithms
- cracking codes and recovering plaintext

Attack types

- **Analysis**: study of the algorithm
-

Cryptanalysis

Cryptanalysis is the study and practice of analyzing and breaking cryptographic security systems to gain access to encrypted information without authorization.

- finding weaknesses in encryption algorithms
- cracking codes and recovering plaintext

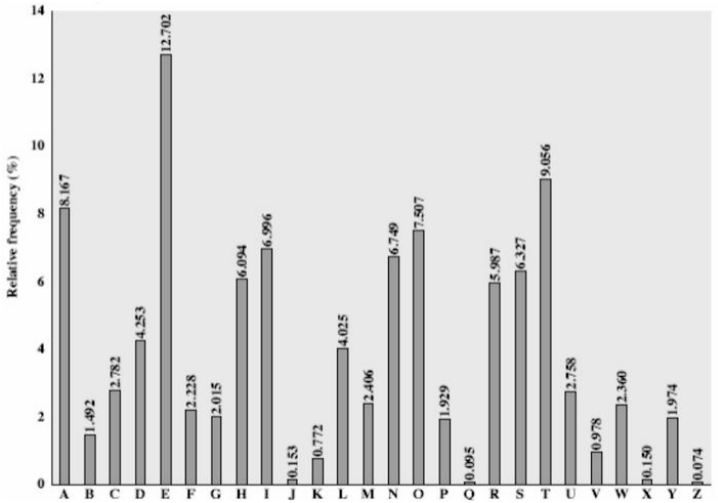
Attack types

- **Analysis**: study of the algorithm
- **Brute-force**: try all possible keys

Monoalphabetic Cipher

Substitution ciphers are vulnerable to frequency analysis attacks.

- In Italian the most frequent character is the character "e". The second most frequent character is the character "a"
- With high probability, in the ciphertext the most frequent character will correspond to the character "e" and the second most frequent character will correspond to the character "a"



Monoalphabetic Cipher

ENGLISH		GERMAN		FINNISH		FRENCH		ITALIAN		SPANISH	
	%		%		%		%		%		%
E	12.31	E	18.46	A	12.06	E	15.87	E	11.79	E	13.15
T	9.59	N	11.42	I	10.59	A	9.42	A	11.74	A	12.69
A	8.05	I	8.02	T	9.76	I	8.41	I	11.28	O	9.49
O	7.94	R	7.14	N	8.64	S	7.90	O	9.83	S	7.60
N	7.19	S	7.04	E	8.11	T	7.26	N	6.88	N	6.95
I	7.18	A	5.38	S	7.83	N	7.15	L	6.51	R	6.25

Code

Julius Caesar

Code - Julius Caesar

```
2  def inrange(n, range_min, range_max):
3      range_len = range_max - range_min + 1
4      a = n % range_len
5      if a < range_min:
6          a = a + range_len
7      return a
8
9  def caesar_shift(c, shift):
10     ascii = ord(c)
11     a = ascii + shift # Now we have a number between 32+shift and 126+shift
12     a = inrange(a, 32, 126) # Put the number back in range
13     return chr(a)
14
15 def caesar_cipher(plaintext, shift):
16     ciphertext = ''
17     for c in plaintext:  # Iterate through the characters of a string
18         ciphertext = ciphertext + caesar_shift(c,shift)
19     return ciphertext
20
```

Code - Julius Caesar

```
22  m = 'Hello! Can you read this?'
23  c = caesar_cipher(m, 6)
24  print('Ciphertext: ', c) # Ciphertext:  Nkrru'&Igt& u{&xkgj&znoyE
25
26  m_2 = caesar_cipher(c, -6)
27  print('Plaintext: ', m_2) # Hello! Can you read this?
28
```

Code

Julius Caesar - Brute Force

Code - Julius Caesar - Brute Force

```
30
31  for i in range(26):
32      print('#', i, ': ', caesar_cipher(c, -i))
33
```

```
# 0 :  Nkrru'&Igt& u{&xkgj&znoyE
# 1 :  Mjqqt&%Hfs%~tz%wjfi%ymnxD
# 2 :  Lipps%$Ger$}sy$vieh$xlmwC
# 3 :  Khoor$#Fdq#|rx#uhdg#wklvB
# 4 :  Jgnnq#"Ecp"{qw"tgcf"vjkuA
# 5 :  Ifmmp"!Dbo!zpv!sfbe!uijt@
# 6 :  Hello! Can you read this?
# 7 :  Gdkkn ~B`m~xnt~qd`c~sghr>
# 8 :  Fcjjm~}A_l}wms}pc_b}rfgq=
# 9 :  Ebiil}|@^k|vlr|ob^a|qefp<
# 10 :  Dahhk|{?]j{ukq{na]`{pdeo;
# 11 :  C`ggj{z>\iztjpzm`\_zocdn:
# 12 :  B_ffizy=[hysioyl_[^ynbcm9
# 13 :  A^eehyx<Zgxrhnxk^Z]xmabl8
# 14 :  @]ddgxw;Yfwqgmwj]Y\wl`ak7
# 15 :  ?\ccfwv:Xevpflvi\X[vk_`j6
# 16 :  >[bbevu9Wduoekuh[WZuj^_i5
# 17 :  =Zaadut8VctndjtgZVYti]^h4
# 18 :  <Y``cts7UbsmcisfYUXsh\]g3
# 19 :  ;X__bsr6TarlbhreXTWrg[\f2
# 20 :  :W^^arq5S`qkagqdWSVqfZ[e1
# 21 :  9V]]`qp4R_pj`fpcVRUpeYZd0
# 22 :  8U\\_po3Q^oi_eobUQTodXYc/
# 23 :  7T[[^on2P]nh^dnaTPSncWXb.
# 24 :  6SZZ]nm1O\mg]cm`SORmbVWa-
# 25 :  5RYY\ml0N[lf\bl_RNQlaUV`,
```

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Challenge

Tutte le strade portano a Roma

Source: https://training.olicyber.it/

Tutte le strade portano a Roma

input: cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}

Cosa sappiamo?

Tutte le strade portano a Roma

input: cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}

Cosa sappiamo?

- la flag inizia con flag{...}

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Tutte le strade portano a Roma

input: cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}

Cosa sappiamo?

- la flag inizia con flag{...}
- flag {--- _ - - - - - - _ - - - - - - - - _ - - _ - - - - - - - -}

a	x
b	
c	
d	
e	
f	c
g	d
h	
i	
j	
k	
l	i
m	

n	
o	
p	
q	
r	
s	
t	
u	
v	
w	
x	
y	
z	

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Tutte le strade portano a Roma

input: cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}

Cosa sappiamo?

- la flag inizia con flag{...}
- flag {a- - _ -a - - a -_ - - - - - - - - -_ - - _ - a l - - a - -}

a	x
b	
c	
d	
e	
f	c
g	d
h	
i	
j	
k	
l	i
m	

n	
o	
p	
q	
r	
s	
t	
u	
v	
w	
x	
y	
z	

Tutte le strade portano a Roma

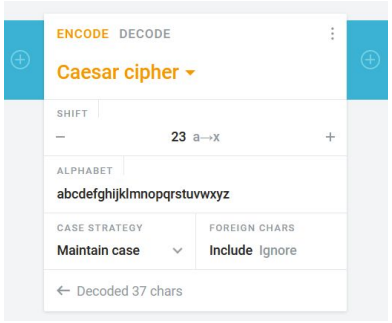
2_crypto > tutte_le_strade_portano_a_roma.py > ...

```python
s = "cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}"

for value in s:
    ascii = ord(value)
    if ascii == 95 or ascii == 123 or ascii == 125:
        print(chr(ascii), end="")
        continue

    ascii = ascii + 3

    if ascii > 122:
        res = ascii % 122
        ascii = 97 + res -1

    print(chr(ascii), end="")

```

2_crypto > tutte_le_strade_portano_a_roma.py > ...

```python
s = "cixd{xsb_zxbpxo_jlofqrof_qb_pxirqxkq}"

for shift in range(26):
    for value in s:
        ascii = ord(value)
        if ascii == 95 or ascii == 123 or ascii == 125:
            print(chr(ascii), end="")
            continue

        ascii = ascii + shift

        if ascii > 122:
            res = ascii % 122
            ascii = 97 + res -1

        print(chr(ascii), end="")
    print("")

```

Tutte le strade portano a Roma



https://cryptii.com/pipes/caesar-cipher

Limitations of Monoalphabetic Cipher

Substitution ciphers have two limitations

- the possible number of shifts is limited
- it is easy to understand if a key is the correct one

Polyalphabetic Cipher

Substitution ciphers are easy to break because they preserve the frequencies of the original alphabet

- Key idea: combine several substitution ciphers

The text is divided into blocks of fixed length and for each block a different substitution is applied for each character

- the key must be as long as the plaintext

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Vigenère Cipher

Message: chebellagioranta

Key: crypto

c	h	e	b	e	l	l	a	g	i	o	r	n	a	t	a
c	r	y	p	t	o	c	r	y	p	t	o	c	r	y	p
e	y	c	q	x	z	n	r	e	x	h	f	p	r	r	p

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
b	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
c	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
d	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
e	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
f	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
g	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
h	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
i	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
j	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
k	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
l	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
m	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
n	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
o	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
p	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
r	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
s	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
t	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
u	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
v	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
w	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
x	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Problem: Repeating the same substitution (alphabet) due to the periodic nature of the key

Secure Encryption

When can we say that a cipher is "secure"?

- a cipher is secure if the ciphertext "appears random," that is, the ciphertext does not leak any information about the plaintext

Given c, c' should be difficult to decide if $m \neq m'$

Threat Models

- **Ciphertext-only attack**: adversary only get one or more cipher texts. Assumed so far.
- **Known-plaintext attack**: adversary gets plaintext/ciphertexts pairs. Must deduce information about other ciphertext
- **Chosen-plaintext attack**: adversary gets plaintext/ciphertext pairs for plaintext of her choice
- **Chosen-ciphertext attack**: adversary additionally may ask for decryptions of cipher texts of her choice

XOR - Exclusive OR

The exclusive OR (XOR) is a binary operator that works on bits (⊕ is the symbol)

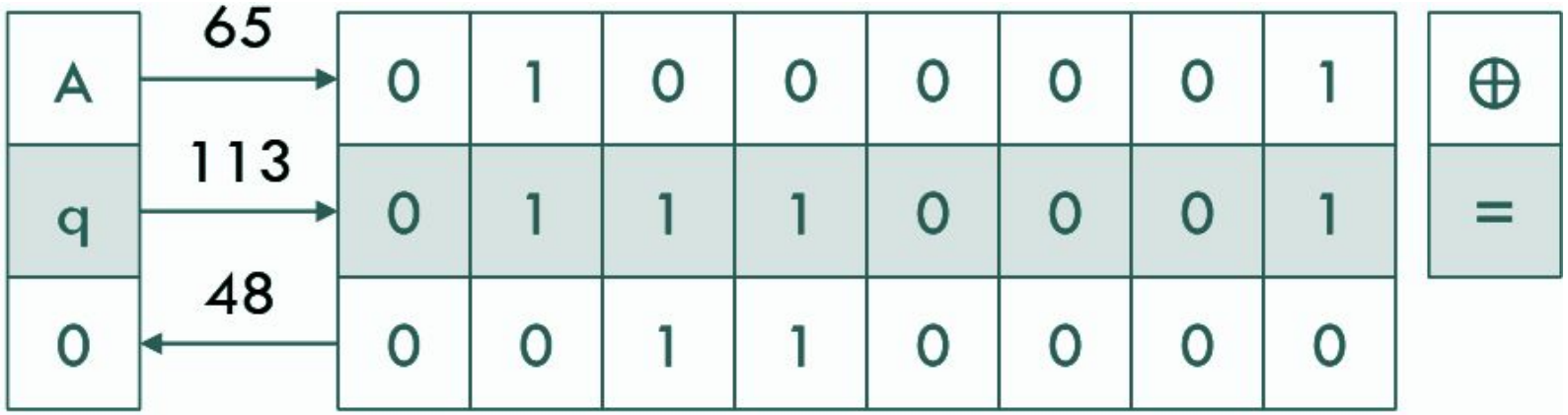
Properties

- $a \oplus (b \oplus c) = (a \oplus b) \oplus c$
- $a \oplus b = b \oplus a$
- $a \oplus a = 0$
- $a \oplus 0 = a$
- $a \oplus b \oplus a = b$

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

XOR - Exclusive OR

Convert the characters from ASCII to binary, calculate the XOR operation vertically and convert the result back



XOR - Exclusive OR

XOR can be used as an encryption algorithm

- m = m1 m2 m3
- k = k1 k2 k3
- c = (m1 $\oplus$ k1) (m2 $\oplus$ k2) (m3 $\oplus$ k3)

XOR encryption is vulnerable to known-plaintext attacks

- given m, c we can retrieve the key as $k = m \oplus c$

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Challenge

Crypto 04 - Xor 1

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 04 - Xor 1

2_crypto > crypto04.py > ...

```python
def xor(a, b):
    return bytes([x^y for x,y in zip(a,b)])

m1 = "158bbd7ca876c60530ee0e0bb2de20ef8af95bc60bdf"
m2 = "73e7dc1bd30ef6576f883e79edaa48dcd58e6aa82aa2"

m1 = bytes.fromhex(m1)
m2 = bytes.fromhex(m2)

res = xor(m1, m2)
print(res)

```

flag: flag{x0R_f0r_th3_w1n!}

Challenge

Crypto 05 - Xor 2

Crypto 05 - Xor 2

2_crypto > crypto05.py > ...

```python
def xor(a, b):
    return bytes([x ^ y for x, y in zip(a, b)])

ciphertext = "104e137f425954137f74107f525511457f5468134d7f146c4c"
ciphertext = bytes.fromhex(ciphertext)

for i in range(128):
    k = chr(i)
    k = str.encode(k) * len(ciphertext)
    plaintext = xor(ciphertext, k)
    print("flag{" + str(plaintext) + "}")

```

flag: flag{0n3_byt3_T0_ru1e_tH3m_4Ll}

OTP - One Time Pad

The one-time pad encryption scheme is perfectly secret

- each bit of the result can be 0 or 1 with equal probability

Operations

- k <- Gen # sampled from a uniform distribution
- $Enc_k(m) = k \oplus m$
- $Dec_k(c) = k \oplus c$

Correctness: $Dec_k(Enc_k(m)) = k \oplus k \oplus m = m$

OTP - One Time Pad

Downsides

- The secret key is as long as the message
- The secret key cannot be reused

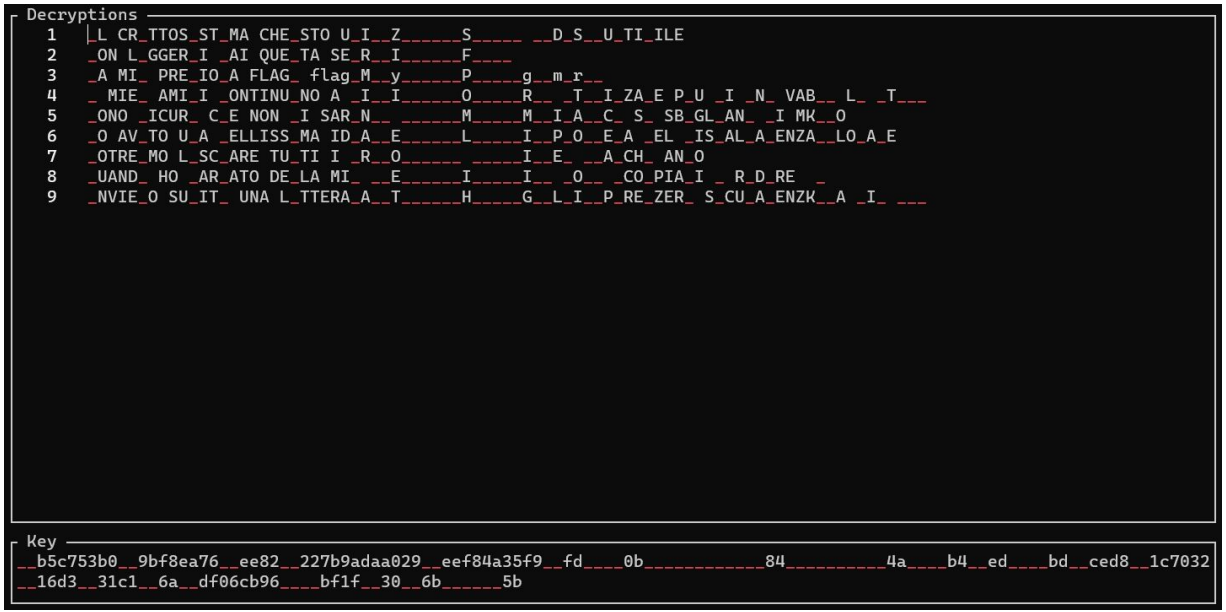
$$Enc_k(m) \oplus Enc_k(m') = m \oplus k \oplus m' \oplus k = m \oplus m'$$

Challenge

Crypto 06 - One more Time Please

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 06 - One more Time Please



flag: flag_M__y_____P_____g__m_r__

[1] https://github.com/CameronLonsdale/MTP

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 06 - One more Time Please

```
Decryptions
  1   IL CRITTOSISTEMA CHE STO UTILIZZANDO SEMBRA INDISTRUTTIBILE
  2   NON LEGGERAI MAI QUESTA SEGRETISSIMA FRASE
  3   LA MIA PREZIOSA FLAG: flag{M4ny_71m3_P4D_N1gH7m4r3}
  4   I MIEI AMICI CONTINUANO A DIRMI CHE NON DOVREI UTILIZZARE PIU DI UNA VOLTA LA STESSA
  5   SONO SICURO CHE NON CI SARANNO PROBLEMI, I MIEI AMICI SI SBAGLIANO DI CERTO
  6   HO AVUTO UNA BELLISSIMA IDEA PER RISOLVERE IL PROBLEMA DEL RISCALDAMENTO GLOBALE
  7   POTREMMO LASCIARE TUTTI I FRIGORIFERI APERTI PER QUALCHE ANNO
  8   QUANDO HO PARLATO DELLA MIA IDEA AI MIEI AMICI SONO SCOPPIATI A RIDERE...
  9   INVIERO SUBITO UNA LETTERA A STEPHEN HAWKING, LUI APPREZZERA SICURAMENTE LA MIA IDEA

Key
78b5c753b08f9bf8ea76a8ee822d227b9adaa02903eef84a35f992fd6b5b0b8548a45f6796841a3453ac594a90d5b4d9ed4b34bdbbced8221c7032
2916d3ca31c1256a12df06c5982d63bf1fdf302c6bb30b0f3a
```

flag: flag{M4ny_71m3_P4D_N1gH7m4r3}

[1] https://github.com/CameronLonsdale/MTP

Challenges

CR_1.01 - Spin Arovnd
CR_1.02 - Planet Matrix
CR_1.03 - Dropped
CR_1.05 - Once Too Many
CR_1.09 - TwoFlags
CR_1.06 - Benchmark